

```

"use client";

import { useState, useId } from "react";
import { CommissionerSettings, DEFAULT_SETTINGS, PoolPlayer, DraftType, MissedCut
import { DEFAULT_FIELD, draftGolfers } from "@lib/pool";

const STEPS = ["Pool Info", "Rules", "Field", "Draft", "Confirm"] as const;

function OptionCard({
  selected, onClick, title, description, badge,
}: {
  selected: boolean; onClick: () => void; title: string; description: string; bad
}) {
  return (
    <button
      type="button"
      onClick={onClick}
      className={`w-full text-left p-4 rounded-lg border-2 transition-all
        ${selected
          ? "border-masters-green bg-masters-green/5"
          : "border-gray-200 hover:border-gray-300 bg-white"}`}
    >
      <div className="flex items-start gap-3">
        <div className={`mt-0.5 w-5 h-5 rounded-full border-2 flex-shrink-0 flex
          ${selected ? "border-masters-green bg-masters-green" : "border-gray-300
          {selected && <div className="w-2 h-2 rounded-full bg-white" />}
        </div>
        <div>
          <div className="flex items-center gap-2">
            <span className="font-semibold text-gray-900 text-sm">{title}</span>
            {badge && (
              <span className="text-[10px] bg-masters-gold text-white px-1.5 py-0
                {badge}
              </span>
            )}
          </div>
          <p className="text-xs text-gray-500 mt-0.5">{description}</p>
        </div>
      </div>
    </button>
  );
}

function Section({ label, children }: { label: string; children: React.ReactNode

```

```

return (
  <div className="mb-8">
    <h3 className="text-xs font-bold uppercase tracking-widest text-gray-400 mb
      {children}
    </div>
  );
}

export default function SetupPage() {
  const uid = useId();
  const [step, setStep] = useState(0);
  const [password, setPassword] = useState("");
  const [authed, setAuthed] = useState(false);
  const [authError, setAuthError] = useState("");

  // Pool info
  const [poolName, setPoolName] = useState("Masters Pool 2026");
  const [buyIn, setBuyIn] = useState(20);
  const [players, setPlayers] = useState<PoolPlayer[]>([
    { id: "p0", name: "" },
  ]);

  // Commissioner settings
  const [settings, setSettings] = useState<CommissionerSettings>(DEFAULT_SETTINGS
  const [customDist, setCustomDist] = useState("100");

  // Golfer field
  const [fieldText, setFieldText] = useState(DEFAULT_FIELD.join("\n"));

  // Draft results
  const [assignments, setAssignments] = useState<ReturnType<typeof draftGolfers>>

  const [saving, setSaving] = useState(false);
  const [saved, setSaved] = useState(false);
  const [error, setError] = useState("");

  const set = <K extends keyof CommissionerSettings>(key: K, val: CommissionerSet
    setSettings((s) => ({ ...s, [key]: val }));

  // — Auth —————
  async function handleAuth(e: React.FormEvent) {
    e.preventDefault();
    setAuthError("");
    const res = await fetch("/api/auth", {
      method: "POST",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify({ password }),

```

```

    });
    if (res.ok) {
        setAuthed(true);
    } else {
        setAuthError("Incorrect password.");
    }
}

// — Players helpers —————
const addPlayer = () =>
    setPlayers((p) => [...p, { id: `p${Date.now()}`, name: "" }]);

const removePlayer = (id: string) =>
    setPlayers((p) => p.filter((pl) => pl.id !== id));

const updatePlayer = (id: string, name: string) =>
    setPlayers((p) => p.map((pl) => (pl.id === id ? { ...pl, name } : pl)));

// — Draft —————
function runDraft() {
    const golferNames = fieldText
        .split("\n")
        .map((n) => n.trim())
        .filter(Boolean);

    const golfers = golferNames.map((name, i) => ({
        id: `g${i}`,
        name,
        r1: null, r2: null, r3: null, r4: null,
        madeCut: null,
    }));

    const result = draftGolfers(players, golfers, settings.draftType);
    setAssignments(result);
    setStep(3);
}

// — Save —————
async function handleSave() {
    setSaving(true);
    setError("");

    const golferNames = fieldText.split("\n").map((n) => n.trim()).filter(Boolean)
    const dist =
        settings.purseType === "custom"
            ? customDist.split(",").map((n) => Number(n.trim()))
            : [];

```

```

const finalSettings = { ...settings, purseDistribution: dist };

const res = await fetch("/api/setup", {
  method: "POST",
  headers: { "Content-Type": "application/json", "x-admin-password": password },
  body: JSON.stringify({
    poolName,
    players: players.filter((p) => p.name.trim()),
    golferNames,
    buyIn,
    settings: finalSettings,
  }),
});

if (res.ok) {
  setSaved(true);
} else {
  setError("Failed to save. Check your connection and try again.");
}
setSaving(false);
}

// — Password gate —————
if (!authenticated) {
  return (
    <div className="max-w-sm mx-auto mt-16">
      <div className="card p-8 text-center">
        <div className="text-4xl mb-4">🔒</div>
        <h2 className="font-serif text-2xl text-masters-green mb-2">Commissioner
        <p className="text-sm text-gray-500 mb-6">Enter your admin password to
        <form onSubmit={handleAuth} className="space-y-4">
          <input
            type="password"
            value={password}
            onChange={(e) => setPassword(e.target.value)}
            placeholder="Password"
            className="w-full border border-gray-300 rounded px-3 py-2 text-sm
          />
          {authError && <p className="text-red-500 text-sm">{authError}</p>}
          <button type="submit" className="btn-green w-full">Enter</button>
        </form>
      </div>
    </div>
  );
}

```

```

// — Saved —————
if (saved) {
  return (
    <div className="max-w-sm mx-auto mt-16 text-center">
      <div className="text-5xl mb-4">🏆</div>
      <h2 className="font-serif text-2xl text-masters-green mb-3">Pool is Live!
      <p className="text-gray-500 mb-6">Share the leaderboard link with your pl
      <a href="/" className="btn-green inline-block">View Leaderboard</a>
    </div>
  );
}

const validPlayers = players.filter((p) => p.name.trim());
const golferCount = fieldText.split("\n").filter((n) => n.trim()).length;

// — Wizard —————
return (
  <div className="max-w-2xl mx-auto">
    {/* Progress */}
    <div className="mb-8">
      <h1 className="font-serif text-3xl text-masters-green mb-4">Commissioner
      <div className="flex items-center gap-0">
        {STEPS.map((label, i) => (
          <div key={label} className="flex items-center flex-1 last:flex-none">
            <div
              className={`flex items-center gap-2 cursor-pointer ${i <= step ?
                onClick={() => i < step && setStep(i)}
            >
              <div
                className={`w-7 h-7 rounded-full flex items-center justify-cent
                  ${i < step ? "bg-masters-gold text-white" : i === step ? "bg-ma
                >
                  {i < step ? "✓" : i + 1}
                </div>
                <span className={`text-xs font-medium hidden sm:block ${i === ste
                  {label}
                </span>
              </div>
            {i < STEPS.length - 1 && (
              <div className={`flex-1 h-0.5 mx-2 ${i < step ? "bg-masters-gold"
            )}
          </div>
        )}}
      </div>
    </div>

    <div className="card p-6">

```

```

{ /* — Step 0: Pool Info ————— */ }
{ step === 0 && (
  <div>
    <h2 className="font-serif text-xl text-masters-green mb-6">Pool Info<

    <Section label="Pool Name">
      <input
        value={poolName}
        onChange={ (e) => setPoolName(e.target.value) }
        className="w-full border border-gray-300 rounded px-3 py-2 text-s
      />
    </Section>

    <Section label="Buy-in Per Player">
      <div className="relative w-40">
        <span className="absolute left-3 top-1/2 -translate-y-1/2 text-gr
        <input
          type="number"
          min={1}
          value={buyIn}
          onChange={ (e) => setBuyIn(Number(e.target.value)) }
          className="w-full border border-gray-300 rounded pl-7 pr-3 py-2
        />
      </div>
    </Section>

    <Section label={`Players (${validPlayers.length})`} >
      <div className="space-y-2">
        {players.map((p) => (
          <div key={p.id} className="flex items-center gap-2">
            <input
              value={p.name}
              onChange={ (e) => updatePlayer(p.id, e.target.value) }
              placeholder="Player name"
              className="flex-1 border border-gray-300 rounded px-3 py-2
            />
            {players.length > 1 && (
              <button
                type="button"
                onClick={() => removePlayer(p.id) }
                className="text-gray-400 hover:text-red-400 transition-co
              >
                <svg className="w-4 h-4" fill="none" viewBox="0 0 24 24"
                  <path strokeLinecap="round" strokeLinejoin="round" stro
                </svg>
              </button>
            )}
          </div>
        )}
      </div>
    </Section>
  </div>
) }

```

```

        })
      </div>
    )))
    <button type="button" onClick={addPlayer} className="text-sm text
      + Add player
    </button>
  </div>

  {validPlayers.length > 0 && (
    <p className="text-xs text-gray-400 mt-3">
      Total purse: <strong className="text-gray-600">${validPlayers.1
    </p>
  )}
</Section>

<div className="flex justify-end">
  <button
    type="button"
    onClick={() => setStep(1)}
    disabled={validPlayers.length < 2}
    className="btn-green disabled:opacity-40"
  >
    Next: Rules →
  </button>
</div>
</div>
))

{/* — Step 1: Rules ————— */}
{step === 1 && (
  <div>
    <h2 className="font-serif text-xl text-masters-green mb-6">Commission

    <Section label="Draft Type">
      <div className="grid grid-cols-1 sm:grid-cols-2 gap-3">
        <OptionCard
          selected={settings.draftType === "snake"}
          onClick={() => set("draftType", "snake")}
          title="🐍 Snake Draft"
          description="Picks reverse each round. Fair distribution across
            badge="Recommended"
        />
        <OptionCard
          selected={settings.draftType === "random"}
          onClick={() => set("draftType", "random")}
          title="🎲 Pure Random"
          description="Golfers assigned sequentially. Simpler, but less b

```

```

    />
  </div>
</Section>

<Section label="Missed Cut Rule">
  <div className="grid grid-cols-1 sm:grid-cols-3 gap-3">
    <OptionCard
      selected={settings.missedCutRule === "penalty"}
      onClick={() => set("missedCutRule", "penalty")}
      title="🚩 Fixed Penalty"
      description="Missed cut golfers receive a set score per remaining hole"
      badge="Recommended"
    />
    <OptionCard
      selected={settings.missedCutRule === "zero"}
      onClick={() => set("missedCutRule", "zero")}
      title="🚫 Zero Contribution"
      description="Missed cut golfers stop counting. No penalty, no bonus"
    />
    <OptionCard
      selected={settings.missedCutRule === "worst-made"}
      onClick={() => set("missedCutRule", "worst-made")}
      title="🏳️ Worst Made Score"
      description="Missed cut golfers receive the total of the worst score"
    />
  </div>

  {settings.missedCutRule === "penalty" && (
    <div className="mt-3 flex items-center gap-3">
      <label className="text-sm text-gray-600">Penalty per missed round</label>
      <div className="flex items-center gap-1">
        <span className="text-sm text-gray-400">+</span>
        <input
          type="number"
          min={1}
          max={20}
          value={settings.missedCutPenalty}
          onChange={(e) => set("missedCutPenalty", Number(e.target.value))}
          className="w-16 border border-gray-300 rounded px-2 py-1.5"
        />
        <span className="text-sm text-gray-400">strokes/round</span>
      </div>
      <span className="text-xs text-gray-400">(x2 rounds = +{settings.missedCutPenalty * 2} strokes)</span>
    </div>
  )}
</Section>

```



```

<Section label="Scoring">
  <div className="grid grid-cols-1 sm:grid-cols-2 gap-3">
    <OptionCard
      selected={settings.scoringType === "all"}
      onClick={() => set("scoringType", "all")}
      title="🏌️ Count All Golfers"
      description="Every golfer on your roster contributes to your to
      badge="Recommended"
    />
    <OptionCard
      selected={settings.scoringType === "best-n"}
      onClick={() => set("scoringType", "best-n")}
      title="⭐ Best N Golfers"
      description="Only your top N performers count. Reduces impact o
    />
  </div>

  {settings.scoringType === "best-n" && (
    <div className="mt-3 flex items-center gap-3">
      <label className="text-sm text-gray-600">Count best:</label>
      <input
        type="number"
        min={1}
        max={20}
        value={settings.bestN}
        onChange={(e) => set("bestN", Number(e.target.value))}
        className="w-16 border border-gray-300 rounded px-2 py-1.5 te
      />
      <span className="text-sm text-gray-400">golfers per player</spa
    </div>
  )}
</Section>

```

```

<Section label="Purse Distribution">
  <div className="grid grid-cols-2 sm:grid-cols-4 gap-3">
    {[
      { value: "winner-take-all" as PurseType, title: "🏆 Winner Take
      { value: "70-30" as PurseType, title: "🥇🥈 70 / 30", desc: "Sf
      { value: "60-30-10" as PurseType, title: "🥇🥈🥉 60/30/10", de
      { value: "custom" as PurseType, title: "✏️ Custom", desc: "Set
    ].map(({ value, title, desc }) => (
      <OptionCard
        key={value}
        selected={settings.purseType === value}
        onClick={() => set("purseType", value)}
        title={title}
        description={desc}
      />
    )
  )}
</div>

```

```

        />
      )))}
    </div>

    {settings.purseType === "custom" && (
      <div className="mt-3">
        <label className="text-xs text-gray-500 block mb-1">
          Comma-separated percentages summing to 100 (e.g. <code>50,30,
        </label>
        <input
          value={customDist}
          onChange={(e) => setCustomDist(e.target.value)}
          placeholder="70,30"
          className="w-48 border border-gray-300 rounded px-3 py-1.5 te
        />
      </div>
    )}
  </Section>

  <div className="flex justify-between">
    <button type="button" onClick={() => setStep(0)} className="text-sm
    <button type="button" onClick={() => setStep(2)} className="btn-gre
  </div>
</div>
)}

{/* — Step 2: Field ————— */}
{step === 2 && (
  <div>
    <h2 className="font-serif text-xl text-masters-green mb-2">Golfer Fie
    <p className="text-sm text-gray-500 mb-4">
      Pre-populated with the 2026 Masters field. Edit as needed — one nam
    </p>

    <textarea
      value={fieldText}
      onChange={(e) => setFieldText(e.target.value)}
      rows={20}
      className="w-full font-mono text-sm border border-gray-300 rounded
    />

    <p className="text-xs text-gray-400 mt-2">
      {golferCount} golfers · {validPlayers.length} players · {" "}
      {golferCount > 0 && validPlayers.length > 0
        ? `~${Math.ceil(golferCount / validPlayers.length)} golfers per p
        : ""}
    </p>

```

```

<div className="flex justify-between mt-4">
  <button type="button" onClick={() => setStep(1)} className="text-sm
  <button
    type="button"
    onClick={runDraft}
    disabled={golferCount < validPlayers.length}
    className="btn-green disabled:opacity-40"
  >
    Run Draft →
  </button>
</div>
</div>
)}

```

```

{/* — Step 3: Draft Results ————— */}
{step === 3 && (
  <div>
    <h2 className="font-serif text-xl text-masters-green mb-2">Draft Resu
    <p className="text-sm text-gray-500 mb-4">
      {settings.draftType === "snake" ? "🐍 Snake draft" : "🎲 Random ass
    </p>

    <div className="space-y-4">
      {validPlayers.map((player) => {
        const myPicks = assignments.filter(a => a.playerId === player.id)
          .sort((a, b) => a.pickNumber - b.pickNumber);
        const golferNames = fieldText.split("\n").map(n => n.trim()).filt
        return (
          <div key={player.id} className="bg-masters-cream rounded-lg p-4
            <div className="font-semibold text-masters-green mb-2 flex it
              <span>{player.name}</span>
              <span className="text-xs text-gray-400 font-normal">{myPick
            </div>
            <div className="flex flex-wrap gap-1.5">
              {myPicks.map((a) => {
                const idx = parseInt(a.golferId.replace("g", ""));
                return (
                  <span key={a.golferId} className="text-xs bg-white bord
                    <span className="text-gray-300 mr-1">#{a.pickNumber}<
                      {golferNames[idx]}
                    </span>
                  </span>
                );
              })}
            </div>
          </div>
        );
      })}
    </div>
  </div>
);

```

```

    }}}
</div>

<div className="flex justify-between mt-6">
  <button
    type="button"
    onClick={() => { runDraft(); }}
    className="text-sm text-masters-green hover:underline"
  >
     Re-run Draft
  </button>
  <div className="flex gap-3">
    <button type="button" onClick={() => setStep(2)} className="text-
    <button type="button" onClick={() => setStep(4)} className="btn-g
  </div>
</div>
</div>
)}

{ /* — Step 4: Confirm ————— */
{step === 4 && (
  <div>
    <h2 className="font-serif text-xl text-masters-green mb-6">Confirm &

    <div className="space-y-3 mb-8">
      {[
        { label: "Pool Name", value: poolName },
        { label: "Players", value: `${validPlayers.length} (${validPlayer
        { label: "Buy-in", value: `${buyIn}/player · ${validPlayers.len
        { label: "Golfers", value: `${fieldText.split("\n").filter(n => n
        {
          label: "Draft", value: settings.draftType === "snake"
            ? "🐍 Snake Draft" : "🎲 Pure Random"
        },
        {
          label: "Missed Cut", value: settings.missedCutRule === "penalty
            ? `🚩 +${settings.missedCutPenalty}/round penalty (+${setting
              : settings.missedCutRule === "zero" ? "🚫 No contribution"
              : "🏆 Worst made-cut score"
        },
        {
          label: "Scoring", value: settings.scoringType === "all"
            ? "👤 All golfers count"
            : `🌟 Best ${settings.bestN} count`
        },
        {
          label: "Purse", value: settings.purseType === "winner-take-all"

```

```

      ? "🏆 Winner take all"
      : settings.purseType === "70-30" ? "70/30 split (top 2)"
      : settings.purseType === "60-30-10" ? "60/30/10 split (top 3)"
      : `Custom: ${customDist}`
    },
  ].map(({ label, value }) => (
    <div key={label} className="flex gap-3 text-sm py-2 border-b bord
      <span className="w-28 text-gray-400 flex-shrink-0">{label}</spa
      <span className="text-gray-800">{value}</span>
    </div>
  )))
</div>

{error && <p className="text-red-500 text-sm mb-4">{error}</p>}}

<div className="flex justify-between">
  <button type="button" onClick={() => setStep(3)} className="text-sm
  <button
    type="button"
    onClick={handleSave}
    disabled={saving}
    className="btn-gold disabled:opacity-60"
  >
    {saving ? "Saving..." : "🏊 Launch Pool"}
  </button>
</div>
</div>
)}
</div>
</div>
);
}

```